

CS4421: DevOps

Week 5: Architecture Evolution, Dynamic Web & App Observability

Instructor: Robert Bordianu

Module: CS4421 - DevOps (8-Week Curriculum)

Primary Stack: JavaScript/Node.js, Astro (SSR), `@astrojs/node`, REST APIs, ADRs

Today's Agenda (3 x 2-Hour Blocks)

- **Block 1 (Quiz + Architecture Evolution):**
 - 🕒 **Quiz 4 (First 15 mins — Locked Single Tab):** Covers Week 4
 - Architecture Evolution: When Static Web (SSG) Hits Its Limits
 - Rendering Paradigms: **SSG vs. SSR vs. Jamstack vs. Serverless**
 - Documenting Decisions: Architecture Decision Records (ADRs)
- **Block 2 (Tooling & Architecture):**
 - Configuring Astro for Standalone Node.js Server Mode
 - Application Observability: Health Probes (Liveness & Readiness)
 - Structured JSON Logging for Production Diagnostics
- **Block 3 (Hands-on Lab):**
 - Authoring `ADR-001` (Transition to SSR & Containers)
 - Building `/api/health` & `/api/feedback` REST Endpoints
 - Running & Testing the Standalone Node.js Server Locally

BLOCK 1: Quiz 4 & Theory

Architecture Evolution & Architecture Decision Records (ADRs)

Quiz 4 Guidelines (15 Minutes)

- **Scope:** Week 4 Cloud Providers, IaC Fundamentals (Terraform vs. CloudFormation), S3, CloudFront, and `git revert` Rollbacks.
- **Integrity Rules:**
 - Strict single-tab lockdown mode via Brightspace.
 - No external tabs, no terminal access, no AI tools.
- **Question Format:**
 - Embedded CloudFront HTTP header responses, S3 policy configurations, and multiple-select questions.
- *When the timer expires, the quiz automatically submits.*

Web Rendering Paradigms Compared

Paradigm	How it Works	Pros	Cons	Best For
SSG (Static)	Pre-renders all HTML at build time	Cheap, fast edge CDN caching, ultra-secure	Rebuild needed for content changes	Blogs, docs, marketing sites
SSR (Server-Rendered)	Renders HTML on-demand per request	Live database access, real-time user auth	Requires running server compute	Dashboards, social feeds
Jamstack / Hybrid	Static shell + client API hydration	Fast initial paint + dynamic widgets	Multiple moving parts	Modern web platforms
Serverless (Functions)	Ephemeral functions (AWS Lambda)	Pay per millisecond, auto-scaling	Cold start latency, execution limits	Event-driven microservices

Documenting Decisions: What is an ADR?

An **Architecture Decision Record (ADR)** is a lightweight markdown document capturing an important architectural choice:

```
# ADR-001: Transition from Static Site to Containerized SSR Runtime
```

```
## Status: Accepted (Date: 2026-09-28)
```

```
## Context
```

```
Our platform requires dynamic API endpoints (/api/health, /api/feedback).  
Static hosting on S3 cannot execute server-side Node.js code.
```

```
## Decision
```

```
We will configure Astro with the `@astrojs/node` adapter in standalone mode  
and package the application as a Docker container.
```

```
## Consequences
```

- Positive: Enables live API routes, dynamic rendering, and operational health checks.
- Negative: Increases operational complexity; requires container compute runtime.

BLOCK 2: Tooling & Architecture

Dynamic Endpoints, Health Probes & Structured Logging

Configuring Astro for Server Mode

By default, Astro outputs static HTML. We install the official Node adapter to convert it to a live HTTP server:

```
npx astro add node
```

In `astro.config.mjs`:

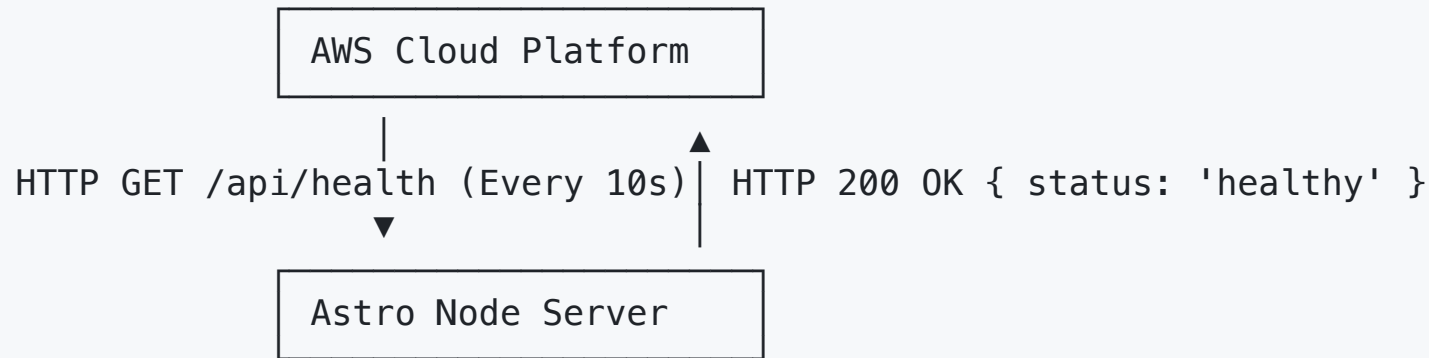
```
import { defineConfig } from 'astro/config';
import node from '@astrojs/node';

export default defineConfig({
  output: 'server', // Enables on-demand server rendering
  adapter: node({
    mode: 'standalone', // Self-contained HTTP server listening on a port
  }),
});
```

- Running `npm run build` now creates `dist/server/entry.mjs` instead of `dist/index.html`.

Application Observability: Health Probes

Production container orchestrators (like AWS App Runner) need automated ways to check if an app is healthy:



- **Liveness Probe:** Is the Node.js process running, or is it deadlocked/crashed?
- **Readiness Probe:** Has the app finished initializing and ready to take user traffic?

Structured JSON Logging vs. `console.log`

In production, human-readable strings like `console.log("user logged in")` are impossible to query across millions of log lines.

Professional DevOps Practice: Structured JSON

```
{  
  "timestamp": "2026-09-28T14:32:01.120Z",  
  "level": "INFO",  
  "route": "/api/feedback",  
  "method": "POST",  
  "statusCode": 200,  
  "latencyMs": 42  
}
```

- Centralized logging tools (Amazon CloudWatch) can query, filter, and alarm on JSON keys instantly (e.g., `filter statusCode >= 500`).

BLOCK 3: Hands-on Practical Lab

Authoring ADR-001 & Building Dynamic Endpoints

Lab 5 Objectives (Individual Repos)

1. Document the Architecture Shift:

- Create `docs/adr/ADR-001-transition-to-ssr-and-containers.md`.

2. Install & Configure the Node.js Adapter:

- Configure `astro.config.mjs` for `output: 'server'`.

3. Build the Operational Health Check Endpoint:

- Create `src/pages/api/health.ts` returning JSON system status:
`{ status: 'ok', uptime: process.uptime(), timestamp: new Date() }`.

4. Implement Structured JSON Logging:

- Create middleware or endpoint logger emitting structured log objects.

5. Run the Standalone Server Locally:

- Build with `npm run build` and execute `node ./dist/server/entry.mjs`.

Step-by-Step Lab Commands

```
# 1. Install official Astro Node adapter
npx astro add node

# 2. Author src/pages/api/health.ts
# export const GET: APIRoute = async () => {
#   return new Response(JSON.stringify({
#     status: 'ok',
#     uptime: process.uptime(),
#     timestamp: new Date().toISOString()
#   }), { status: 200, headers: { 'Content-Type': 'application/json' } });
# }

# 3. Compile standalone server build
npm run build

# 4. Run the compiled Node.js server
HOST=0.0.0.0 PORT=4321 node ./dist/server/entry.mjs

# 5. In a second terminal, verify with curl:
curl -i http://localhost:4321/api/health
```

Week 5 Deliverables & Checkpoint

- ☐ `docs/adr/ADR-001-transition-to-ssr-and-containers.md` committed.
- ☐ Astro configured in `output: 'server'` mode with `@astrojs/node`.
- ☐ Dynamic `/api/health` endpoint responding with HTTP 200 and JSON status.
- ☐ Structured JSON logging visible in terminal during incoming requests.

Reminder for Next Week:

Quiz 5 takes place in the **first 15 minutes of Week 6 Block 1** (Locked Single Tab).

Covers Week 5 Rendering Paradigms, ADRs, Dynamic Endpoints, Health Probes, and Structured Logging.